

2.5 API Gateway: put your Lambda on the internet

i This is the point where the pieces you have built start becoming a **real backend API**.

Your architecture will become:

Postman / Next.js



API Gateway



Lambda



DynamoDB



Lambda



API Gateway



JSON response

Learning object

REST API vs HTTP API in API Gateway — what's the difference, and which would you pick for a new service?

They are both types of RESTful APIs. The difference being REST is just an updated version of HTTP APIs, introducing new commands like DELETE and many other things [Choose between](#)

[REST APIs and HTTP APIs - Amazon API Gateway](#) (link to a page that shows all the differences between the two APIs).

However, even though REST APIs are more feature-rich, I believe this project is better suited to HTTP APIs. This is because they are cheaper and a lot simpler to use. Additionally, the task only requires making endpoints for GET and POST, which HTTP APIs support. So there is no real need for a REST API here.

What are a route, a resource and a stage?

Route = A route is a **HTTP method + a path** e.g. GET + /branches/{branchId}/readings. API Gateway chooses the matching route when a request arrives.

There are three routes that we need currently for this project:

Resource = These are like endpoints or components that your API connects to, such as a Lambda or a database. e.g., an API connecting to a Lambda (resource), or a Lambda connecting to a database (resource).

AI DEF on internet:

- The data or object you use.
- Noun-based and digital.
- Represents real-world things like files or users.
- Accessed or changed via the route.

Stage = A stage is your environment. For example, with the APIs you are working with you have links to your API 'stage' which Postman uses to access the API you want to work with.

AI DEF on internet:

- A step in the release process.
- Keeps testing separate from live use.
- Controls configuration per environment.
- Examples: Dev, Test, Prod

How does a path parameter get from the URL into your Lambda's event?

~~What is a path parameter = is a variable part of a web address or URL used in web applications and APIs to point to a specific item or resource. It sits directly inside the URL path separated by slashes, such as /users/42, where 42 is the path parameter for a specific user.~~

After creating the API in AWS I realised the paragraph above was unclear and Luke's explanation is more accurate.



Luke explanation: So by path parameter it's your queries. For example:

GET <https://api.com/animals/cats?age=2> (Would return all cats in your database where their age field is 2).

How would we use this to get from the URL to the Lambda event? =

What is CORS? Read this properly now — it will bite you in Week 4 and you'll be glad you did

Short DEF: Cross-Origin Resource Sharing (CORS) is a security mechanism built into web browsers. It lets a server declare whether other websites can request its data. It works using special HTTP headers. By default, browsers block these cross-domain requests to keep user data safe.



Luke explanation: It basically allows access for the API to communicate between different places, e.g., front-end communicating with backend. It uses headers to store information such as the API key to access API Gateway in AWS.

Specifically allows communication between a web app and an API.

Task



This task is about learning how to create endpoints, then making sure they work in Postman (doing the GET and POST requests in Postman).

- `{{GET /branches/{branchId}/readings}}` — that branch's readings, newest first. Then add optional `{{from}}` and `{{to}}` query parameters
- `{{GET /branches/{branchId}/readings/{recordedAt}}}` — one reading
- `{{POST /branches/{branchId}/readings}}` — create a reading. This one validates the body, computes `{{status}}` using your function from 1.7, generates the `{{readingId}}`, and saves it

Then prove it from Postman. Save a collection with a request for each of these:

- A valid in-range reading → 201, and `{{status}}` comes back as `{{ok}}`
- A valid out-of-range reading → 201, and `{{status}}` comes back as `{{excursion}}`
- `{{maxTempC}}` below `{{minTempC}}` → 400
- A missing required field → 400
- A timestamp in the future → 400
- The same reading twice → 409 the second time

- A reading that doesn't exist → 404
- A branch with no readings → 200 and an empty array

Okay in this section we are working in three places:

AWS Lambda → write the backend code

API Gateway → create the three HTTP endpoints

Postman → call the endpoints and prove they work

Your DynamoDB table is already the database behind the Lambda. We can reuse the Lambda that already has DynamoDB permissions rather than starting again.

The finished path will be:

```
1 Postman
2   ↓
3 API Gateway
4   ↓
5 Lambda
6   ↓
7 DynamoDB
8   ↓
9 Lambda
10  ↓
11 API Gateway
12  ↓
13 Postman response
```

AWS Lambda

First: I had to replace the lambda `umar_coldchain_reading` with this JavaScript, because we are reusing this lambda instead of creating a completely new one. This saves time on the setup.

```
1 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
2
3 import {
4   DynamoDBDocumentClient,
5   PutCommand,
6   QueryCommand
7 } from "@aws-sdk/lib-dynamodb";
8
9 import { randomUUID } from "node:crypto";
10
11 // -----
12 // DYNAMODB SETUP
13 // -----
14
15 const client = new DynamoDBClient({
16   region: "eu-north-1"
17 });
18
19 const dynamo = DynamoDBDocumentClient.from(client);
20
21 const TABLE_NAME = "umar_coldchain_database";
```

```

23
24
25 // -----
26 // BUSINESS RULE FROM TASK 1.7
27 // -----
28
29 function getReadingStatus(minTempC, maxTempC) {
30
31     if (!Number.isFinite(minTempC) || !Number.isFinite(maxTempC)) {
32         throw new TypeError(
33             "minTempC and maxTempC must both be valid numbers"
34         );
35     }
36
37     if (minTempC < 2 || maxTempC > 8) {
38         return "excursion";
39     }
40
41     return "ok";
42 }
43
44
45 // -----
46 // HTTP RESPONSE HELPER
47 // -----
48
49 function response(statusCode, body) {
50
51     return {
52         statusCode: statusCode,
53
54         headers: {
55             "content-type": "application/json"
56         },
57
58         body: JSON.stringify(body)
59     };
60 }
61
62
63 // -----
64 // CONVERT DATE TO STANDARD ISO FORMAT
65 // -----
66
67 function normaliseDate(value) {
68
69     const decoded = decodeURIComponent(value);
70
71     const timestamp = Date.parse(decoded);
72
73     if (Number.isNaN(timestamp)) {
74         return null;
75     }
76
77     return new Date(timestamp).toISOString();
78 }
79
80
81 // -----
82 // FIND READING BY TIMESTAMP
83 // -----
84
85 async function findReadingByTimestamp(branchId, recordedAt) {
86
87     const result = await dynamo.send(
88         new QueryCommand({
89

```

```

90     TableName: TABLE_NAME,
91
92     KeyConditionExpression:
93         "branchId = :branchId AND begins_with(readingKey,
:recordedAt)",
94
95     ExpressionAttributeValues: {
96         ":branchId": branchId,
97         ":recordedAt": `${recordedAt}#`
98     },
99
100     Limit: 1
101 })
102 );
103
104 return result.Items?.[0];
105 }
106
107
108 // -----
109 // LAMBDA HANDLER
110 // -----
111
112 export const handler = async (event, context) => {
113
114     console.log("API request received", {
115         requestId: context.awsRequestId,
116         routeKey: event.routeKey,
117         pathParameters: event.pathParameters,
118         queryParameters: event.queryStringParameters
119     });
120
121     try {
122
123         const route = event.routeKey;
124
125         const branchId =
126             event.pathParameters?.branchId;
127
128
129         // =====
130         // GET ALL READINGS FOR A BRANCH
131         // =====
132
133         if (
134             route ===
135             "GET /branches/{branchId}/readings"
136         ) {
137
138             const from =
139                 event.queryStringParameters?.from;
140
141             const to =
142                 event.queryStringParameters?.to;
143
144
145             let keyCondition =
146                 "branchId = :branchId";
147
148             const values = {
149                 ":branchId": branchId
150             };
151
152
153             // FROM AND TO PROVIDED
154             if (from && to) {

```

```

156
157     const normalisedFrom =
158         normaliseDate(from);
159
160     const normalisedTo =
161         normaliseDate(to);
162
163     if (!normalisedFrom || !normalisedTo) {
164         return response(400, {
165             error: "from and to must be valid dates"
166         });
167     }
168
169     if (
170         new Date(normalisedFrom) >
171         new Date(normalisedTo)
172     ) {
173         return response(400, {
174             error: "from must be before to"
175         });
176     }
177
178     keyCondition +=
179         " AND readingKey BETWEEN :from AND :to";
180
181     values[":from"] =
182         `${normalisedFrom}#`;
183
184     values[":to"] =
185         `${normalisedTo}#\uffff`;
186 }
187
188
189 // ONLY FROM PROVIDED
190 else if (from) {
191
192     const normalisedFrom =
193         normaliseDate(from);
194
195     if (!normalisedFrom) {
196         return response(400, {
197             error: "from must be a valid date"
198         });
199     }
200
201     keyCondition +=
202         " AND readingKey >= :from";
203
204     values[":from"] =
205         `${normalisedFrom}#`;
206 }
207
208
209 // ONLY TO PROVIDED
210 else if (to) {
211
212     const normalisedTo =
213         normaliseDate(to);
214
215     if (!normalisedTo) {
216         return response(400, {
217             error: "to must be a valid date"
218         });
219     }
220
221     keyCondition +=
222         " AND readingKey <= :to";

```

```

223     values[":to"] =
224         `${normalisedTo}#\uffff`;
225     }
226
227
228
229     const result = await dynamo.send(
230         new QueryCommand({
231
232             TableName: TABLE_NAME,
233
234             KeyConditionExpression:
235                 keyCondition,
236
237             ExpressionAttributeValues:
238                 values,
239
240             ScanIndexForward: false
241         })
242     );
243
244
245     return response(
246         200,
247         result.Items ?? []
248     );
249 }
250
251
252 // =====
253 // GET ONE READING
254 // =====
255
256 if (
257     route ===
258     "GET /branches/{branchId}/readings/{recordedAt}"
259 ) {
260
261     const recordedAt =
262         normaliseDate(
263             event.pathParameters.recordedAt
264         );
265
266
267     if (!recordedAt) {
268
269         return response(400, {
270             error: "recordedAt must be a valid date"
271         });
272     }
273
274
275     const reading =
276         await findReadingByTimestamp(
277             branchId,
278             recordedAt
279         );
280
281
282     if (!reading) {
283
284         return response(404, {
285             error: "Reading not found"
286         });
287     }
288
289

```

```

290     return response(
291         200,
292         reading
293     );
294 }
295
296
297 // =====
298 // POST / CREATE READING
299 // =====
300
301 if (
302     route ===
303     "POST /branches/{branchId}/readings"
304 ) {
305
306     let body;
307
308     try {
309
310         body = JSON.parse(
311             event.body ?? "{}"
312         );
313     }
314
315     catch {
316
317         return response(400, {
318             error: "Request body must be valid JSON"
319         });
320     }
321
322
323 // -----
324 // REQUIRED FIELDS
325 // -----
326
327 const requiredFields = [
328     "recordedAt",
329     "minTempC",
330     "maxTempC",
331     "recordedBy"
332 ];
333
334
335 for (const field of requiredFields) {
336
337     if (
338         body[field] === undefined ||
339         body[field] === null ||
340         body[field] === ""
341     ) {
342
343         return response(400, {
344             error:
345             `Missing required field: ${field}`
346         });
347     }
348 }
349
350
351 // -----
352 // TEMPERATURE VALIDATION
353 // -----
354
355 if (
356     !Number.isFinite(body.minTempC) ||

```

```

357     !Number.isFinite(body.maxTempC)
358   ) {
359
360     return response(400, {
361       error:
362         "minTempC and maxTempC must be numbers"
363     });
364   }
365
366   if (
367     body.maxTempC <
368     body.minTempC
369   ) {
370
371     return response(400, {
372       error:
373         "maxTempC must be greater than or equal to minTempC"
374     });
375   }
376 }
377
378 // -----
379 // STATUS MUST BE SERVER OWNED
380 // -----
381
382 if (body.status !== undefined) {
383
384   return response(400, {
385     error:
386       "status is calculated by the server and must not be
387 supplied"
388   });
389 }
390
391 // -----
392 // DATE VALIDATION
393 // -----
394
395 const recordedAt =
396   normaliseDate(
397     body.recordedAt
398   );
399
400
401 if (!recordedAt) {
402
403   return response(400, {
404     error:
405       "recordedAt must be a valid date"
406   });
407 }
408
409
410 if (
411   new Date(recordedAt) >
412   new Date()
413 ) {
414
415   return response(400, {
416     error:
417       "recordedAt cannot be in the future"
418   });
419 }
420
421
422

```

```

423 // -----
424 // CHECK FOR DUPLICATE
425 // -----
426
427 const existing =
428   await findReadingByTimestamp(
429     branchId,
430     recordedAt
431   );
432
433
434 if (existing) {
435
436   return response(409, {
437     error:
438       "A reading already exists for this branch and timestamp"
439   });
440 }
441
442
443 // -----
444 // CALCULATE STATUS
445 // -----
446
447 const status =
448   getReadingStatus(
449     body.minTempC,
450     body.maxTempC
451   );
452
453
454 // -----
455 // GENERATE READING ID
456 // -----
457
458 const readingId =
459   randomUUID();
460
461
462 const readingKey =
463   `${recordedAt}#${readingId}`;
464
465
466 // -----
467 // BUILD DYNAMODB ITEM
468 // -----
469
470 const item = {
471
472   branchId: branchId,
473
474   readingKey: readingKey,
475
476   readingId: readingId,
477
478   recordedAt: recordedAt,
479
480   minTempC: body.minTempC,
481
482   maxTempC: body.maxTempC,
483
484   recordedBy: body.recordedBy,
485
486   status: status
487 };
488
489

```

```

490 // -----
491 // SAVE READING
492 // -----
493
494 await dynamo.send(
495     new PutCommand({
496
497         TableName: TABLE_NAME,
498
499         Item: item
500     })
501 );
502
503
504 console.log(
505     "Reading created",
506     {
507         branchId,
508         readingId,
509         status
510     }
511 );
512
513
514 return response(
515     201,
516     item
517 );
518 }
519
520
521 // =====
522 // UNKNOWN ROUTE
523 // =====
524
525 return response(404, {
526     error: "Route not found"
527 });
528
529
530 } catch (error) {
531
532     console.error(
533         "Unhandled API error",
534         error
535     );
536
537
538     return response(500, {
539         error:
540             "Internal server error"
541     });
542 }
543 };

```

I know this code looks like a lot but basically, this code makes one Lambda handle all three endpoints at once.

For example: GET /branches/BRANCH-001/readings.

The Lambda performs a DynamoDB **Query** for: branchId = BRANCH-001.

and: ScanIndexForward: false

gives you newest first.

It returns: 200 OK.

API Gateway

Now we are configuring the API Gateway:

- **i** Creating the GET, POST and HTTP APIs in API Gateway:
 - `{{GET /branches/{branchId}/readings}}` — that branch's readings, newest first. Then add optional `{{from}}` and `{{to}}` query parameters
 - `{{GET /branches/{branchId}/readings/{recordedAt}}` — one reading
 - `{{POST /branches/{branchId}/readings}}` — create a reading. This one validates the body, computes `{{status}}` using your function from 1.7, generates the `{{readingId}}`, and saves it

First we have to create our API Gateway.

We go to API Gateway on the Lambda console → Create API → HTTP API → Build.

Then I added the Lambda to the integration (added the specific lambda I needed, Umar_coldchain_LinkDynamoDB) and created that API.

Now we are creating the routes:

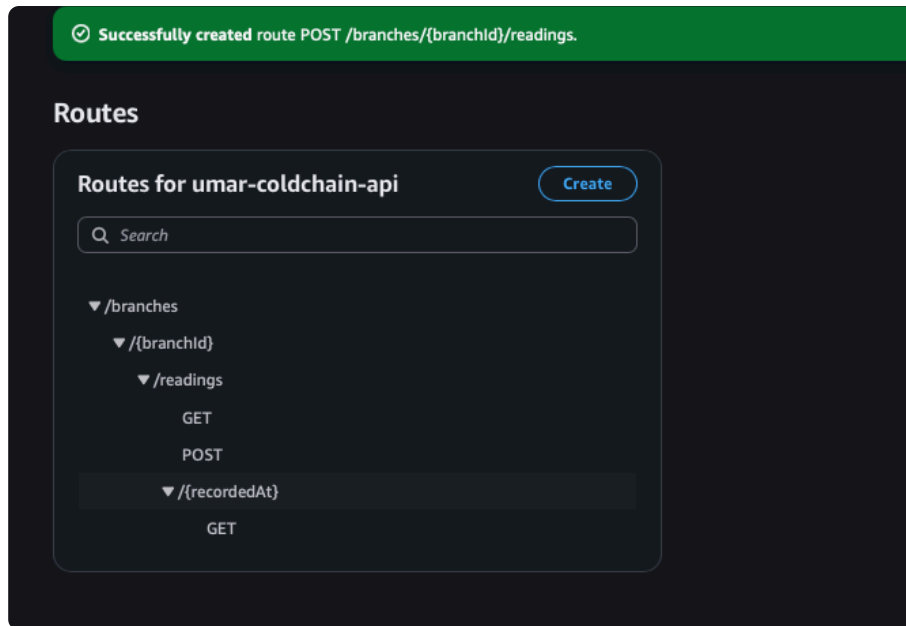
I got a bit confused on this part.

Route 1 = GET /branches/{branchId}/readings

Route 2 = GET /branches/{branchId}/readings/{recordedAt}

Route 3 = POST /branches/{branchId}/readings

After adding all the routes to a single API Gateway it looks like this:



🤔 **CONFUSED** on what's happening??

Basically, from my understanding API Gateway combines the HTTP method and route path to match incoming requests.

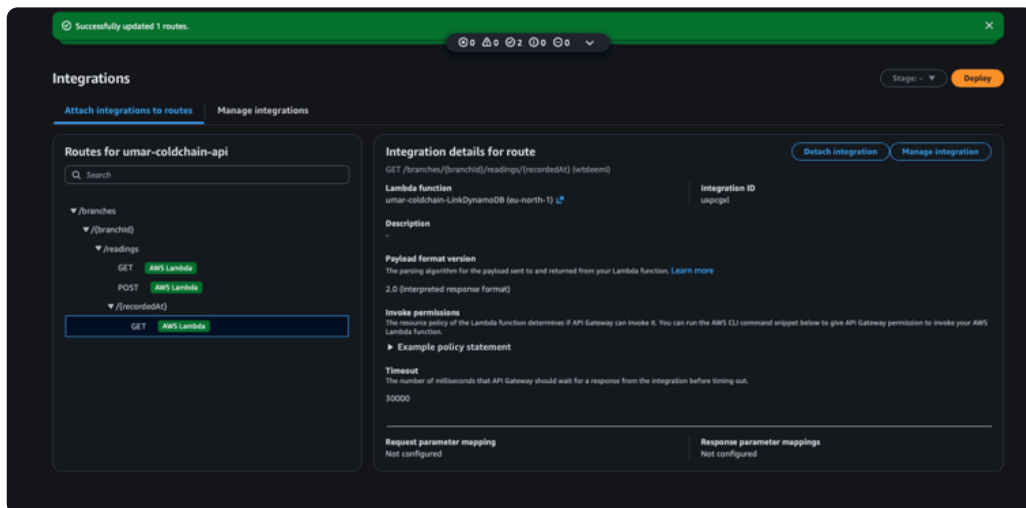
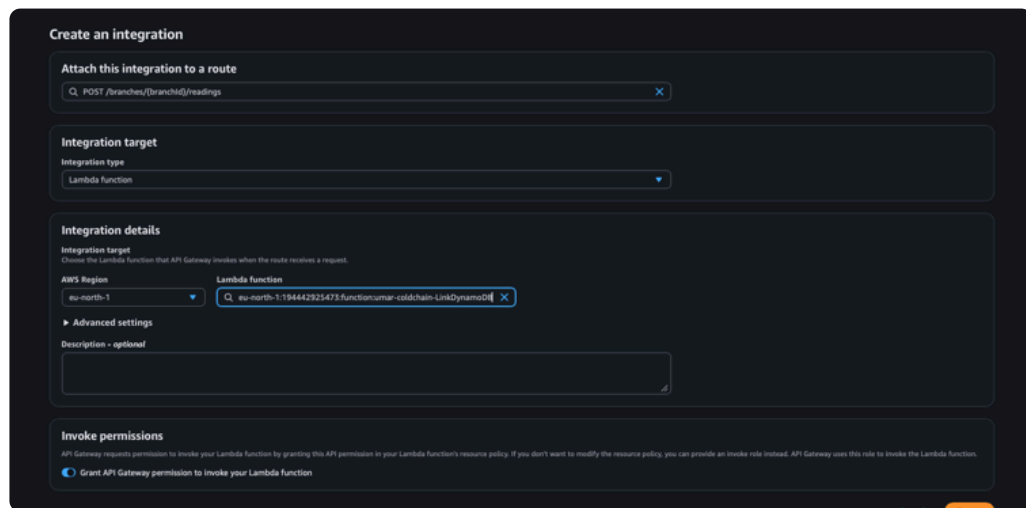
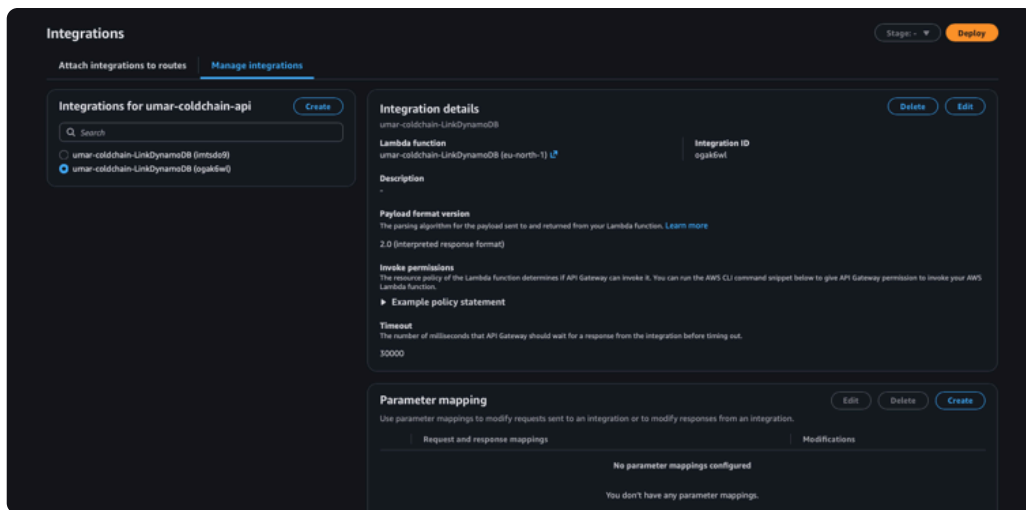
I was confused why it doesn't actually have branch id in the API name, but it makes sense not to, because if an API request came in asking for branch-001 the route design will match and return the correct branch because of the path structure.

Link from stage API section and configuring Lambda

Okay now we are getting the link from the stage in the API which Postman will be using and we are connecting the Lambda to each route so it has access to DynamoDB.

FIRST we got this link from the \$default in the stage section of the API (important as Postman will be using it) = <https://gsbx9vph3g.execute-api.eu-north-1.amazonaws.com/>

THEN I went and configured the Umar-coldchain-LinkDynamoDB to the API and ensured it was connected to the three routes so they all had access to it. Shown below:

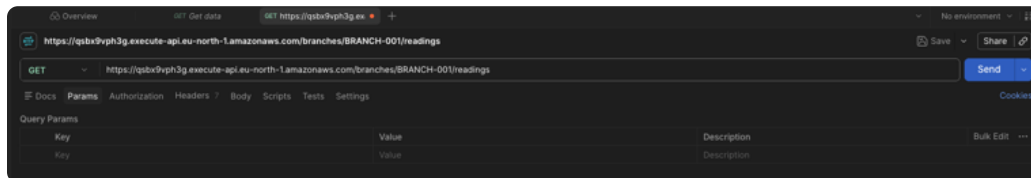


Postman testing

i Now we are running the GET and POST requests to get the following results:

- A valid in-range reading → 201, and `{{status}}` comes back as `{{ok}}`
- A valid out-of-range reading → 201, and `{{status}}` comes back as `{{excursion}}`
- `{{maxTempC}}` below `{{minTempC}}` → 400
- A missing required field → 400
- A timestamp in the future → 400
- The same reading twice → 409 the second time
- A reading that doesn't exist → 404
- A branch with no readings → 200 and an empty array

FIRST: 'A branch with no readings → 200 and an empty array'



So first I opened Postman and selected a new HTTP GET request. Then I input the following URL: <https://qsbx9vph3g.execute-api.eu-north-1.amazonaws.com/branches/BRANCH-001/readings>

Then I clicked send.

ERROR 500 = When send was clicked I received a “500 Internal Server Error”.

First I checked the Lambda Umar-coldchain-LinkDynamoDB CloudWatch Logs and it showed that it was not a permissions problem on the side of the API Gateway.

The issue was that the Lambda was still using the **old test-event code** from the previous DynamoDB ticket. That code expects events like:

```
{
  "action": "queryBranch",
  "branchId": "BRANCH-001"
}
```

But API Gateway doesn't send an `action` property. With HTTP API payload version 2.0, API Gateway sends information such as `routeKey`, `pathParameters`, query parameters and the request body instead.

Therefore it was changed to:

```
1 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
2
3 import {
4   DynamoDBDocumentClient,
5   PutCommand,
6   QueryCommand
7 } from "@aws-sdk/lib-dynamodb";
8
9 import { randomUUID } from "node:crypto";
10
11
12 // -----
13 // DYNAMODB SETUP
14 // -----
15
16 const client = new DynamoDBClient({
17   region: "eu-north-1"
18 });
19
20 const dynamo = DynamoDBDocumentClient.from(client);
21
22 const TABLE_NAME = "umar_coldchain_database";
23
24
25 // -----
26 // BUSINESS RULE FROM TASK 1.7
27 // -----
28
29 function getReadingStatus(minTempC, maxTempC) {
30
31   if (!Number.isFinite(minTempC) || !Number.isFinite(maxTempC))
32   {
33     throw new TypeError(
34       "minTempC and maxTempC must both be valid numbers"
35     );
36   }
37
38   if (minTempC < 2 || maxTempC > 8) {
39     return "excursion";
40   }
41
42   return "ok";
43 }
44
45 // -----
46 // HTTP RESPONSE HELPER
47 // -----
48
49 function response(statusCode, body) {
50
51   return {
52     statusCode: statusCode,
53
54     headers: {
55       "content-type": "application/json"
```

```

56     },
57
58     body: JSON.stringify(body)
59   };
60 }
61
62
63 // -----
64 // CHECK AND NORMALISE DATE
65 // -----
66
67 function normaliseDate(value) {
68
69   const timestamp = Date.parse(value);
70
71   if (Number.isNaN(timestamp)) {
72     return null;
73   }
74
75   return new Date(timestamp).toISOString();
76 }
77
78
79 // -----
80 // FIND ONE READING USING ITS TIMESTAMP
81 // -----
82
83 async function findReadingByTimestamp(branchId, recordedAt) {
84
85   const result = await dynamo.send(
86     new QueryCommand({
87       TableName: TABLE_NAME,
88
89       KeyConditionExpression:
90         "branchId = :branchId AND begins_with(readingKey,
91         :recordedAt)",
92
93       ExpressionAttributeValues: {
94         ":branchId": branchId,
95         ":recordedAt": `${recordedAt}#`
96       },
97
98       Limit: 1
99     })
100   );
101   return result.Items?.[0];
102 }
103
104
105 // -----
106 // LAMBDA HANDLER
107 // -----
108
109 export const handler = async (event, context) => {
110
111   console.log("API request received", {
112     requestId: context.awsRequestId,
113     routeKey: event.routeKey,
114     pathParameters: event.pathParameters,
115     queryParameters: event.queryStringParameters
116   });
117
118
119   try {
120
121     const route = event.routeKey;

```

```

122
123     const branchId =
124         event.pathParameters?.branchId;
125
126
127     // =====
128     // ROUTE 1
129     // GET /branches/{branchId}/readings
130     //
131     // Gets every reading for one branch.
132     // Results are newest first.
133     // Also supports optional from and to dates.
134     // =====
135
136     if (
137         route ===
138         "GET /branches/{branchId}/readings"
139     ) {
140
141         const from =
142             event.queryStringParameters?.from;
143
144         const to =
145             event.queryStringParameters?.to;
146
147
148         let keyCondition =
149             "branchId = :branchId";
150
151
152         const values = {
153             ":branchId": branchId
154         };
155
156
157         // -----
158         // FROM AND TO ARE BOTH PROVIDED
159         // -----
160
161         if (from && to) {
162
163             const normalisedFrom =
164                 normaliseDate(from);
165
166             const normalisedTo =
167                 normaliseDate(to);
168
169
170             if (!normalisedFrom || !normalisedTo) {
171
172                 return response(400, {
173                     error: "from and to must be valid dates"
174                 });
175             }
176
177
178             if (
179                 new Date(normalisedFrom) >
180                 new Date(normalisedTo)
181             ) {
182
183                 return response(400, {
184                     error: "from must be before to"
185                 });
186             }
187
188

```

```

189         keyCondition +=
190             " AND readingKey BETWEEN :from AND :to";
191
192
193         values[":from"] =
194             `${normalisedFrom}#`;
195
196         values[":to"] =
197             `${normalisedTo}#\uffff`;
198     }
199
200
201     // -----
202     // ONLY FROM IS PROVIDED
203     // -----
204
205     else if (from) {
206
207         const normalisedFrom =
208             normaliseDate(from);
209
210
211         if (!normalisedFrom) {
212
213             return response(400, {
214                 error: "from must be a valid date"
215             });
216         }
217
218
219         keyCondition +=
220             " AND readingKey >= :from";
221
222
223         values[":from"] =
224             `${normalisedFrom}#`;
225     }
226
227
228     // -----
229     // ONLY TO IS PROVIDED
230     // -----
231
232     else if (to) {
233
234         const normalisedTo =
235             normaliseDate(to);
236
237
238         if (!normalisedTo) {
239
240             return response(400, {
241                 error: "to must be a valid date"
242             });
243         }
244
245
246         keyCondition +=
247             " AND readingKey <= :to";
248
249
250         values[":to"] =
251             `${normalisedTo}#\uffff`;
252     }
253
254
255     const result = await dynamo.send(

```

```

256         new QueryCommand({
257             TableName: TABLE_NAME,
258
259             KeyConditionExpression:
260                 keyCondition,
261
262             ExpressionAttributeValues:
263                 values,
264
265             // False means newest readings first.
266             ScanIndexForward: false
267         })
268     };
269
270
271     // If the branch has no readings,
272     // return 200 with an empty array.
273     return response(
274         200,
275         result.Items ?? []
276     );
277 }
278
279
280
281 // =====
282 // ROUTE 2
283 // GET /branches/{branchId}/readings/{recordedAt}
284 //
285 // Gets one reading using its timestamp.
286 // =====
287
288 if (
289     route ===
290     "GET /branches/{branchId}/readings/{recordedAt}"
291 ) {
292
293     const recordedAt =
294         normaliseDate(
295             event.pathParameters?.recordedAt
296         );
297
298
299     if (!recordedAt) {
300
301         return response(400, {
302             error: "recordedAt must be a valid date"
303         });
304     }
305
306
307     const reading =
308         await findReadingByTimestamp(
309             branchId,
310             recordedAt
311         );
312
313
314     if (!reading) {
315
316         return response(404, {
317             error: "Reading not found"
318         });
319     }
320
321
322     return response(

```

```

323         200,
324         reading
325     );
326 }
327
328
329
330 // =====
331 // ROUTE 3
332 // POST /branches/{branchId}/readings
333 //
334 // Creates a new reading.
335 // =====
336
337 if (
338     route ===
339     "POST /branches/{branchId}/readings"
340 ) {
341
342     let body;
343
344
345     // -----
346     // PARSE JSON BODY
347     // -----
348
349     try {
350
351         body = JSON.parse(
352             event.body ?? "{}"
353         );
354
355     } catch {
356
357         return response(400, {
358             error: "Request body must be valid JSON"
359         });
360     }
361
362
363     // -----
364     // REQUIRED FIELDS
365     // -----
366
367     const requiredFields = [
368         "recordedAt",
369         "minTempC",
370         "maxTempC",
371         "recordedBy"
372     ];
373
374
375     for (const field of requiredFields) {
376
377         if (
378             body[field] === undefined ||
379             body[field] === null ||
380             body[field] === ""
381         ) {
382
383             return response(400, {
384                 error:
385                     `Missing required field: ${field}`
386             });
387         }
388     }
389

```

```

390
391 // -----
392 // TEMPERATURES MUST BE NUMBERS
393 // -----
394
395 if (
396     !Number.isFinite(body.minTempC) ||
397     !Number.isFinite(body.maxTempC)
398 ) {
399
400     return response(400, {
401         error:
402             "minTempC and maxTempC must be numbers"
403     });
404 }
405
406
407 // -----
408 // MAXIMUM CANNOT BE BELOW MINIMUM
409 // -----
410
411 if (
412     body.maxTempC <
413     body.minTempC
414 ) {
415
416     return response(400, {
417         error:
418             "maxTempC must be greater than or equal to
minTempC"
419     });
420 }
421
422
423 // -----
424 // CLIENT SHOULD NOT SET STATUS
425 // -----
426
427 if (body.status !== undefined) {
428
429     return response(400, {
430         error:
431             "status is calculated by the server and must not be
supplied"
432     });
433 }
434
435
436 // -----
437 // CHECK TIMESTAMP
438 // -----
439
440 const recordedAt =
441     normaliseDate(
442         body.recordedAt
443     );
444
445
446 if (!recordedAt) {
447
448     return response(400, {
449         error:
450             "recordedAt must be a valid date"
451     });
452 }
453
454

```

```

455 // Timestamp cannot be in the future.
456 if (
457     new Date(recordedAt) >
458     new Date()
459 ) {
460
461     return response(400, {
462         error:
463             "recordedAt cannot be in the future"
464     });
465 }
466
467
468 // -----
469 // CHECK FOR DUPLICATE
470 // -----
471
472 const existing =
473     await findReadingByTimestamp(
474         branchId,
475         recordedAt
476     );
477
478
479 if (existing) {
480
481     return response(409, {
482         error:
483             "A reading already exists for this branch and
timestamp"
484     });
485 }
486
487
488 // -----
489 // WORK OUT STATUS
490 // -----
491
492 const status =
493     getReadingStatus(
494         body.minTempC,
495         body.maxTempC
496     );
497
498
499 // -----
500 // GENERATE UNIQUE READING ID
501 // -----
502
503 const readingId =
504     randomUUID();
505
506
507 // Sort key:
508 // timestamp + reading ID
509 const readingKey =
510     `${recordedAt}#${readingId}`;
511
512
513 // -----
514 // CREATE DYNAMODB ITEM
515 // -----
516
517 const item = {
518
519     branchId: branchId,
520

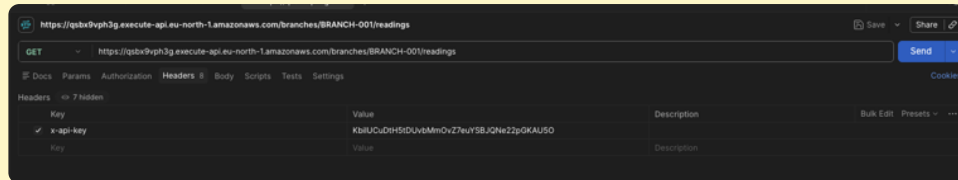
```

```

521         readingKey: readingKey,
522
523         readingId: readingId,
524
525         recordedAt: recordedAt,
526
527         minTempC: body.minTempC,
528
529         maxTempC: body.maxTempC,
530
531         recordedBy: body.recordedBy,
532
533         status: status
534     };
535
536
537     // -----
538     // SAVE TO DYNAMODB
539     // -----
540
541     await dynamo.send(
542         new PutCommand({
543             TableName: TABLE_NAME,
544             Item: item
545         })
546     );
547
548
549     console.log("Reading created", {
550         requestId: context.awsRequestId,
551         branchId: branchId,
552         readingId: readingId,
553         status: status
554     });
555
556
557     // 201 = successfully created
558     return response(
559         201,
560         item
561     );
562 }
563
564
565
566     // =====
567     // NO MATCHING ROUTE
568     // =====
569
570     return response(404, {
571         error: "Route not found"
572     });
573
574
575 } catch (error) {
576
577     // Unexpected problem.
578     console.error(
579         "Unhandled API error",
580         error
581     );
582
583
584     return response(500, {
585         error: "Internal server error"
586     });
587 }

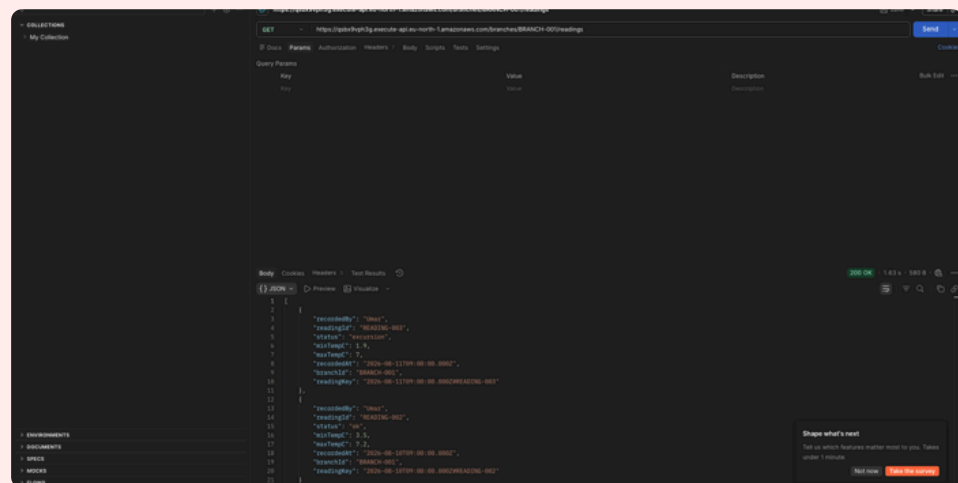
```

⚠️ **ERROR (medium error)** = I was told by Luke that the error might be occurring because of not including the API key in the header. However he said that on the assumption that it was a RESTful API whereas I made an HTTP API so I do not need this. I removed it.

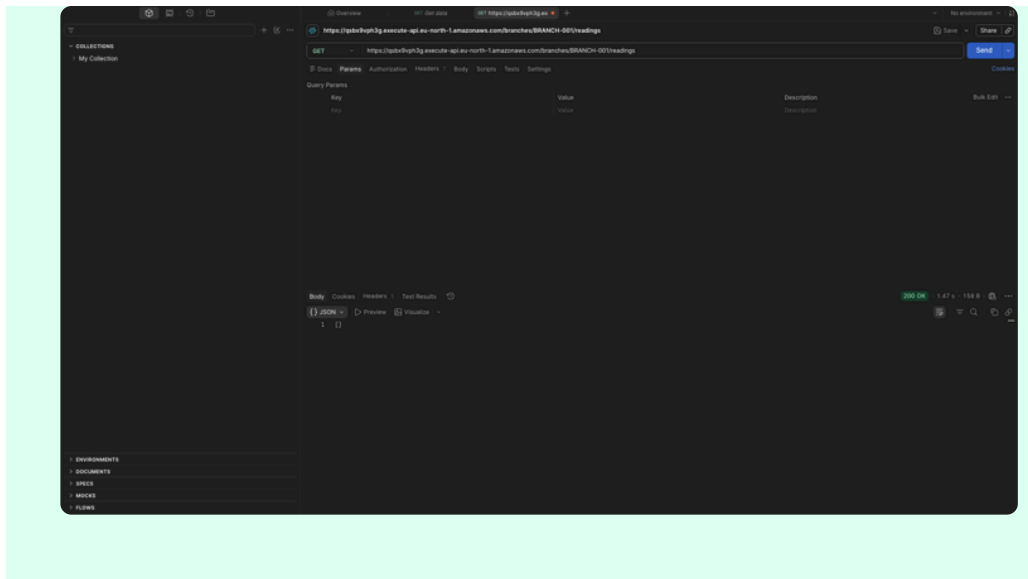


After testing it with Luke just now it works with and without an API key. However it's bad practice not to include one, so start using them.

❌ **LOGIC ERROR** = It passed with 200 OK: the GET all readings in branch-001 worked; however it pulled two readings that were in the database. This is not an issue with the API — it shows the API request is working — but the task specifies the database needs to be emptied. As a result I have labelled this a logic error: nothing is wrong with the code but the database needs to be cleared.



✅ **PASSED** = finally the GET was run and it passed with 200 OK and no values as [] was returned.

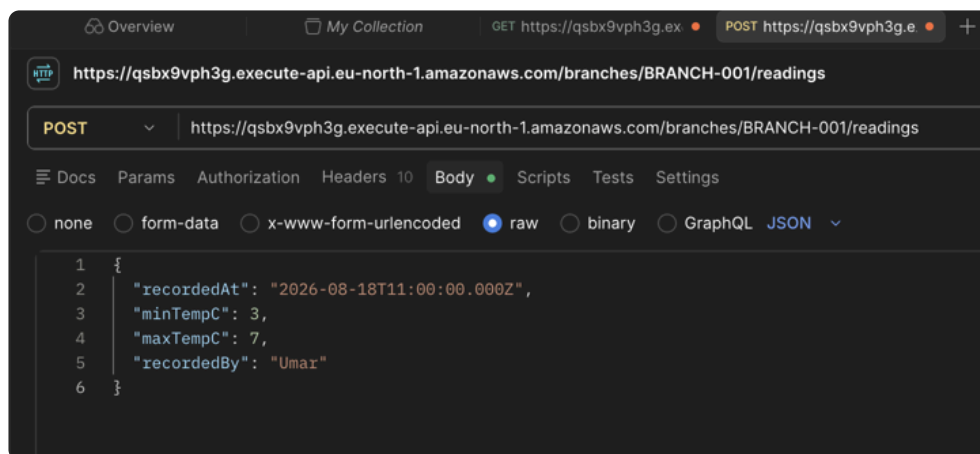


SECOND: A valid in-range reading → 201, and {{status}} comes back as {{ok}}.

I ran into one problem when doing this test; however it wasn't a system/syntax error. Nothing was wrong with the code or the infrastructure — it was human error on my part.

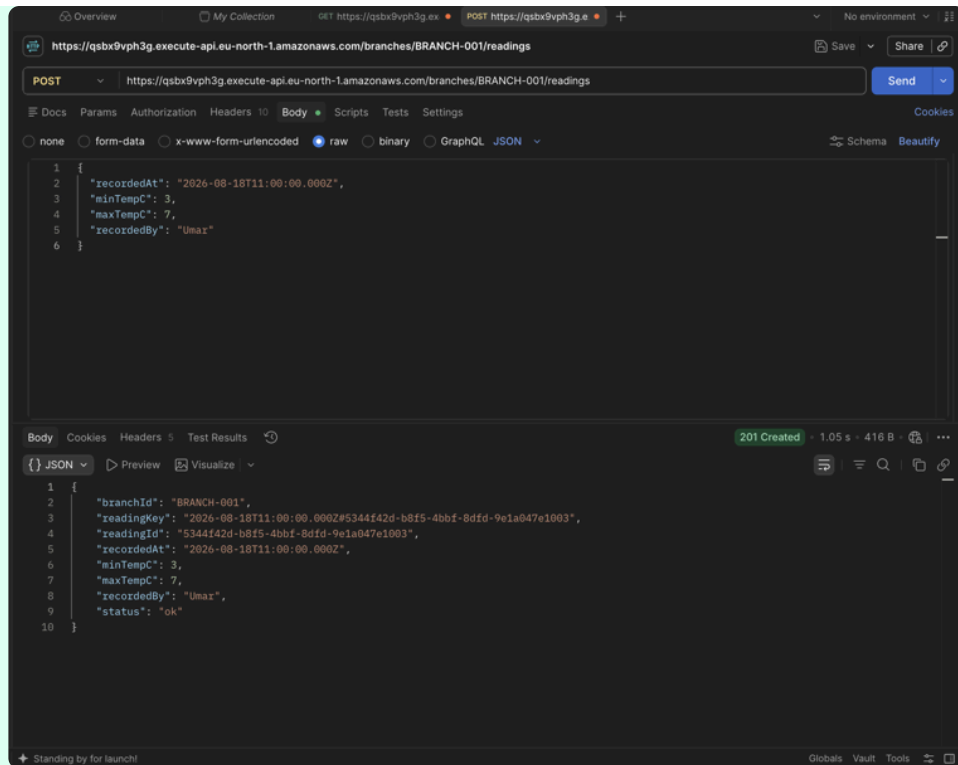
The reason was I was conducting a POST to check a value. I input the correct URL but I forgot to put the JSON body in the request. The body contains the values that are actually being checked, which is why it didn't work.

The JSON code I had to input is shown below:



When the JSON body was included the request passed with a result of OK because the two temps were within the expected range.

✓ **PASSED** = it passed with 201 Created and the response body showed the result OK.

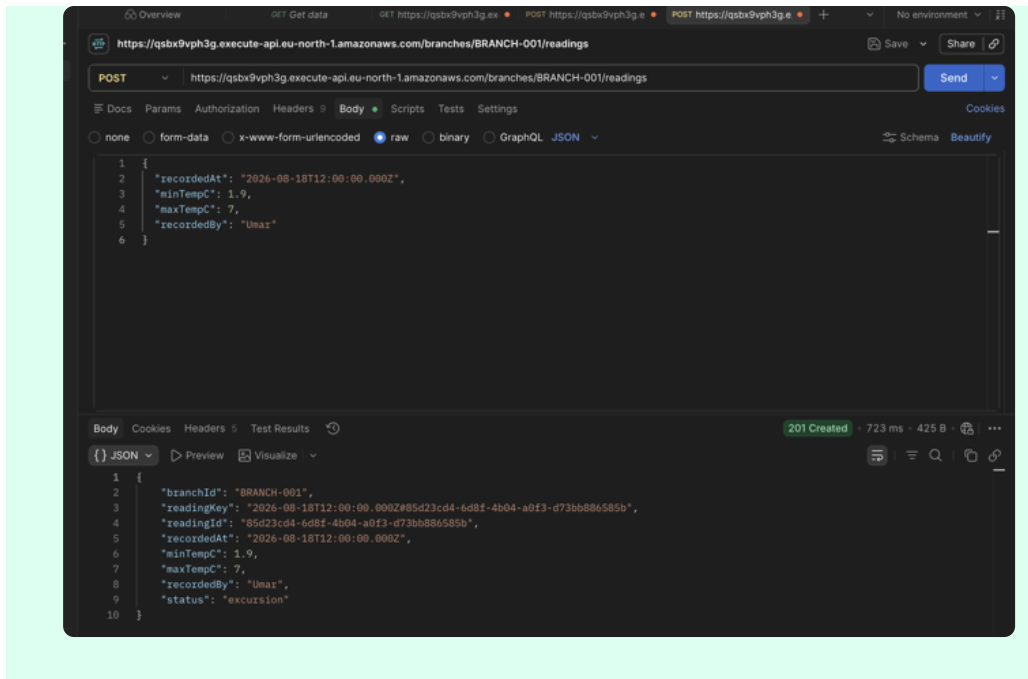


THIRD: A valid out-of-range reading → 201, and {{status}} comes back as {{excursion}}.

Same as the one above; the only difference is we changed the values in the JSON body to be outside the accepted range to get a response of 'excursion'.

```
1 {
2   "recordedAt": "2026-08-18T12:00:00.000Z",
3   "minTempC": 1.9,
4   "maxTempC": 7,
5   "recordedBy": "Umar"
6 }
```

✓ **PASSED** = it passed with 201 Created and the response body showed the result of 'excursion'.



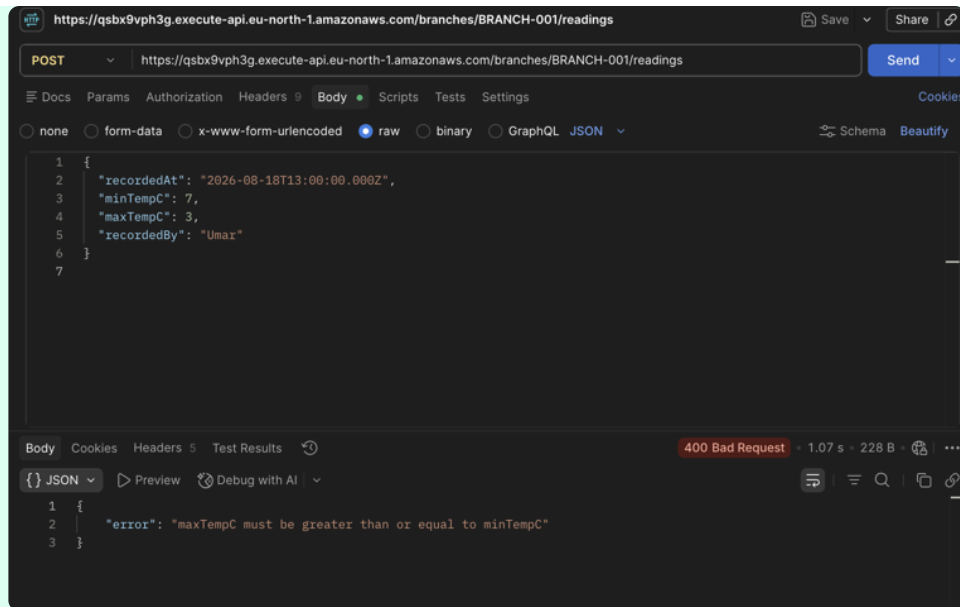
FOURTH: “{{maxTempC}} below {{minTempC}} → 400”

To get this result I input the code below into the body.

```
1 {
2   "recordedAt": "2026-08-18T13:00:00.000Z",
3   "minTempC": 7,
4   "maxTempC": 3,
5   "recordedBy": "Umar"
6 }
7
```

This is a 'Bad Request' because max and min temps are switched and do not make logical sense.

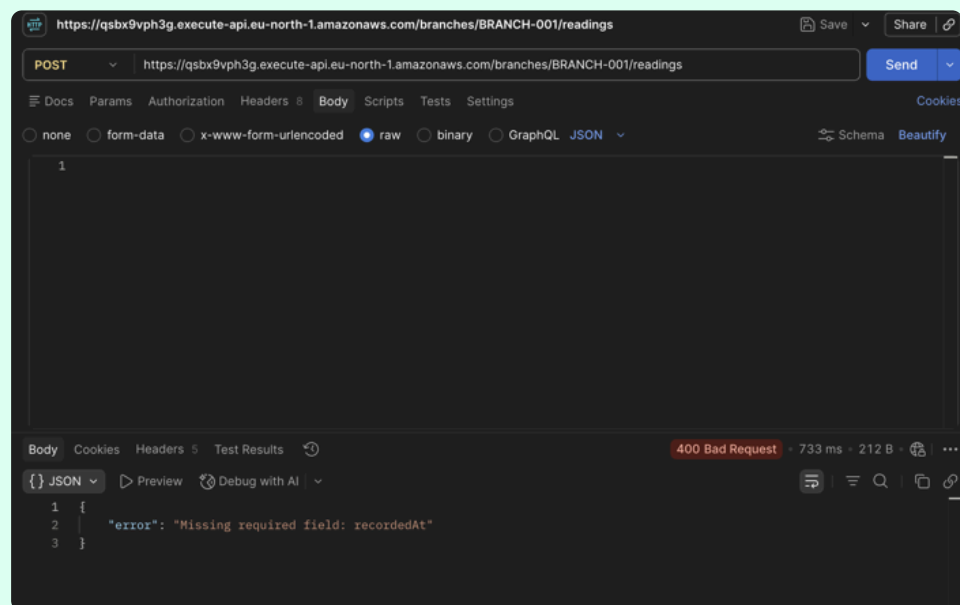
✓ **PASSED** = it returned 400 Bad Request as expected and the response body said "maxTempC must be greater than or equal to minTempC".



FIFTH: “A missing required field → 400”

I managed to get the missing required field error accidentally when I made the error on the SECOND test. I received the 400 missing field by not including the JSON body in the request; hence the request was missing a required field.

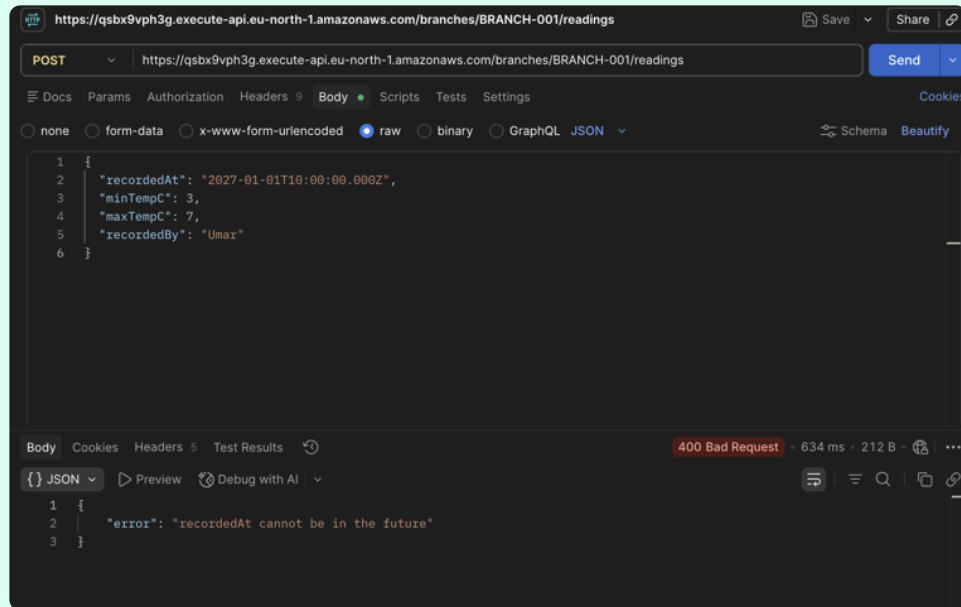
✓ **PASSED** = it returned 400 Bad Request as expected, and the response message said missing required field.



SIXTH: "A timestamp in the future → 400"

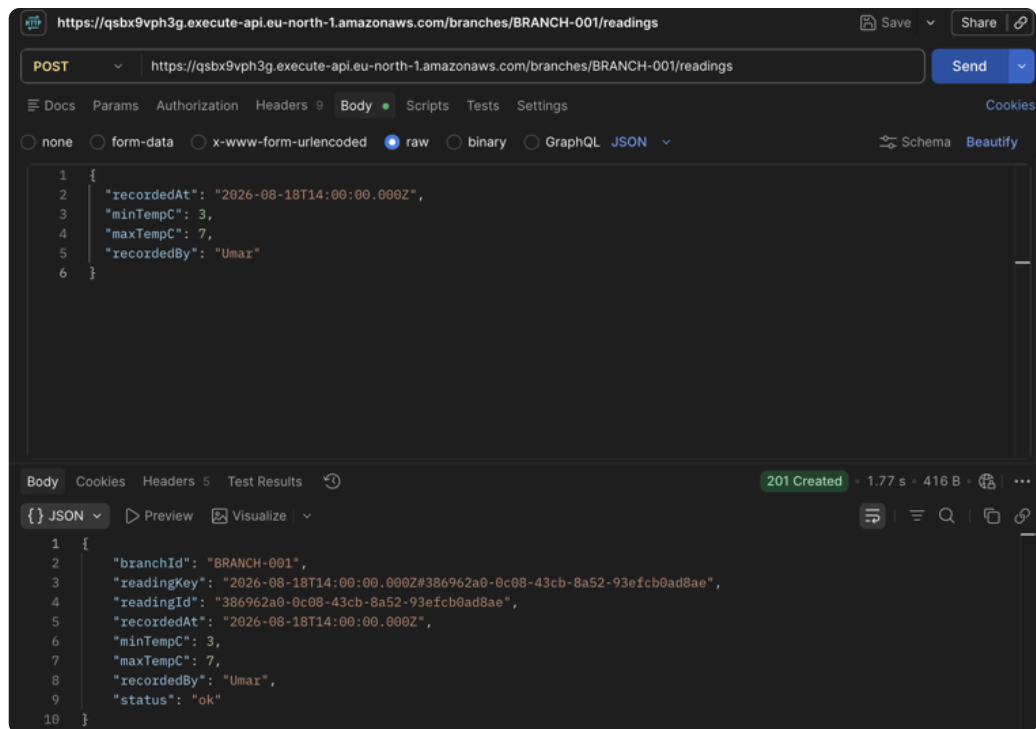
To get this result I changed the recorded timestamp to 2027 and as a result I got 400 Bad Request with the expected message.

✓ **PASSED** = it returned 400 Bad Request as expected, and the message in the response said 'recordedAt cannot be in the future'.



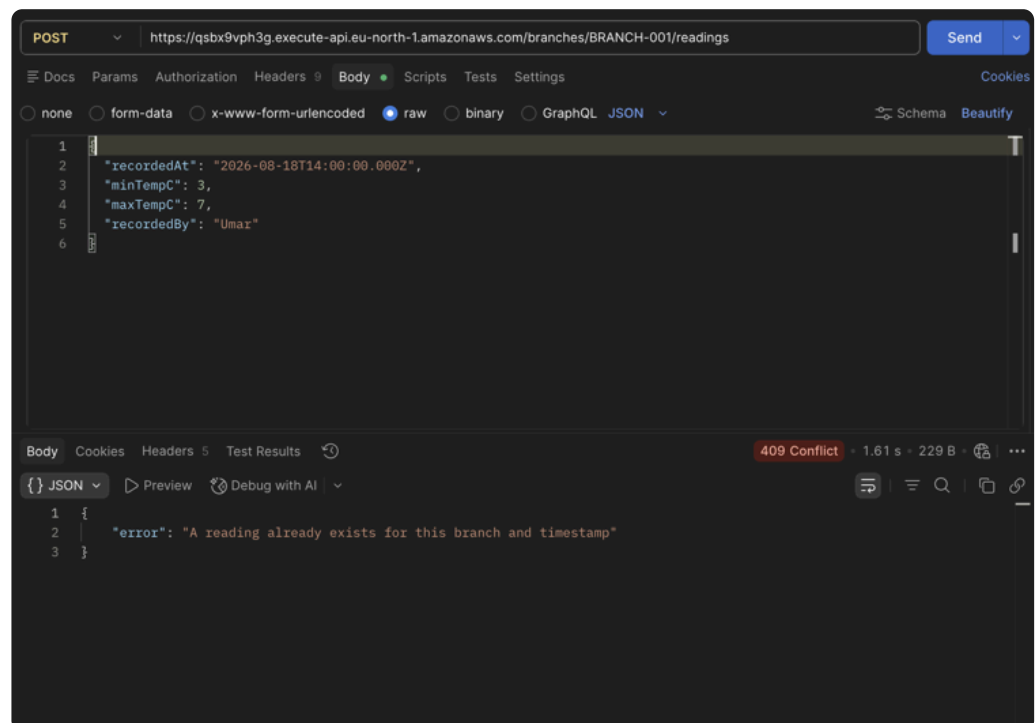
SEVENTH: "The same reading twice → 409 the second time"

For this one I first had to send this request:



This returned 201 because the reading did not exist yet.

Then I clicked send again with the exact same request.



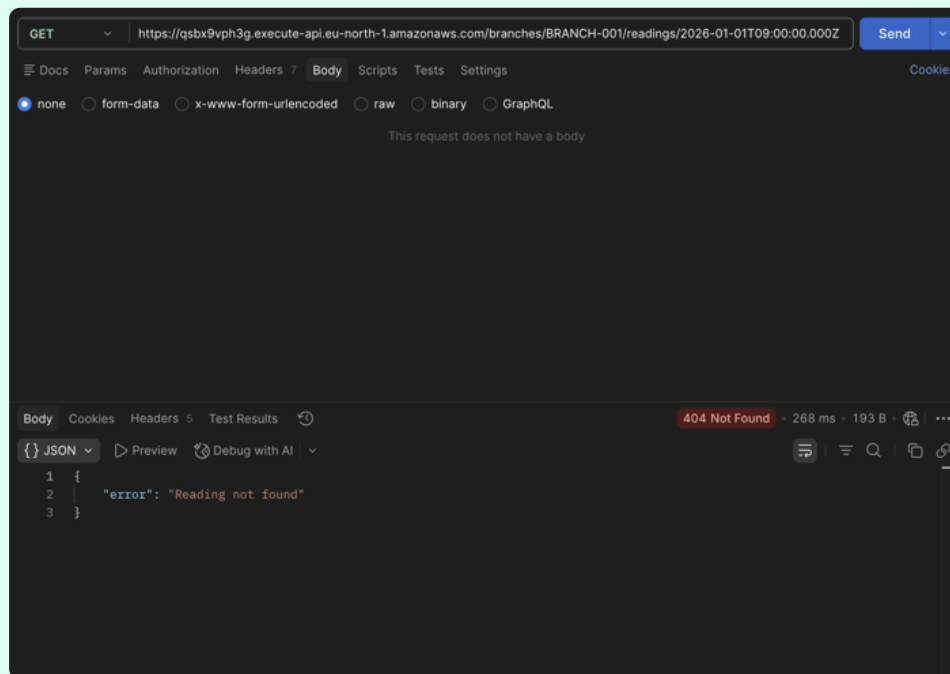
and got the 409 Conflict we were looking for.

EIGHTH: "A reading that doesn't exist → 404"

To get this result we did a GET to fetch a timestamp that doesn't exist in the database to get 404 Not Found.

We fetched: <https://qsbx9vph3g.execute-api.eu-north-1.amazonaws.com/branches/BRANCH-001/readings/2026-01-01T09:00:00.000Z>

✓ **PASSED** = it returned 404 Not Found and the response body contained the message "Reading not found" after fetching <https://qsbx9vph3g.execute-api.eu-north-1.amazonaws.com/branches/BRANCH-001/readings/2026-01-01T09:00:00.000Z>.



Help learn

[How to Create a CRUD API – NodeJS and Express Project for Beginners](#)

Read to learn how to make an API.

[RESTful APIs in 100 Seconds // Build an API from Scratch with Node.js Express](#)

Watch the second half of the video to see how to make a RESTful API in Node.js.

 Questions / Queries / Pushback

I don't understand this at all; I am just getting more and more confused.

The four questions I don't understand:

- **route, a resource and a stage?**
- **How does a path parameter get from the URL into your Lambda's event?**

The API Gateway stuff I am just lost on; I don't understand it.

I understand it better now but my brain is fried and I have spent so long on this section.